



Sesi 3: State Management & Local Data



Overview Sesi

Durasi: 90 menit (1.5 jam) | **Branch:** 05-dart-basics → 06-controller-state → 07-in-memory-crud | **Tujuan:** Belajar Dart fundamentals, Controllers, dan implementasi CRUD in-memory



Agenda Sesi

Bagian 1: Dart Fundamentals (25 menit)

- ✓ Variables & data types
- ✓ Lists & Maps
- ✓ Functions
- ✓ Null safety basics

Bagian 2: Controllers & State Management (35 menit)

- ✓ Generate Controller dengan Metro CLI
- ✓ Understand Controller lifecycle
- ✓ setState() untuk update UI
- ✓ Connect Controller ke Page

Bagian 3: In-Memory CRUD Operations (30 menit)

- ✓ Create task (add to list)
 - ✓ Read tasks (display from list)
 - ✓ Update task (edit properties)
 - ✓ Delete task (remove from list)
-



Bagian 1: Dart Fundamentals (25 menit)

Sebelum masuk ke state management, kita perlu pahami dasar-dasar bahasa Dart dulu.

1.1 Variables & Data Types

Variables adalah tempat menyimpan data. Di Dart, ada beberapa cara declare variable:

```
// 1. var - tipe data otomatis terdeteksi
var name = 'Budi'; // String
var age = 25;      // int
var height = 175.5; // double

// 2. Explicit type - lebih jelas
String name = 'Budi';
int age = 25;
double height = 175.5;
bool isStudent = true;

// 3. final - nilai tidak bisa diubah setelah diset
final String email = 'budi@example.com';
// email = 'new@email.com'; // ERROR!

// 4. const - constant compile-time (lebih strict dari final)
const int maxUsers = 100;

// 5. dynamic - tipe data bisa berubah-ubah (jarang dipakai)
dynamic value = 'Hello';
value = 123;    // OK! Bisa ganti ke int
value = true;   // OK! Bisa ganti ke bool
value = [1, 2, 3]; // OK! Bisa ganti ke List
```

Kapan pakai apa? - var: Untuk variable yang nilainya bisa berubah (tipe tetap) - final: Untuk variable yang diset sekali saja - const: Untuk nilai constant yang sudah diketahui saat compile - dynamic: Ketika tipe data bisa berubah-ubah (⚠️ avoid jika memungkinkan, karena tidak type-safe)

⚠️ Kenapa hindari dynamic?

Type safety = Dart tahu tipe data dari setiap variable, sehingga bisa detect error saat coding (sebelum run app).

Contoh masalah dengan dynamic:

```
// ❌ Dengan dynamic - Error baru ketahuan saat app running
dynamic username = 'Budi';
username = 123; // OK compile, tapi logically wrong!
print(username.toUpperCase()); // CRASH! int tidak punya method
                             toUpperCase()
```

```
// ✅ Dengan type jelas - Error langsung terdeteksi saat coding
String username = 'Budi';
username = 123; // ERROR saat coding: "Can't assign int to String"
print(username.toUpperCase()); // Safe!
```

Best practice: Pakai var atau explicit type (String, int, dll) untuk dapat benefit type safety!

1.2 Lists (Array)

List adalah kumpulan data yang teratur. Index dimulai dari 0.

```
// Create list
List<String> fruits = ['Apple', 'Banana', 'Orange'];

// Atau pakai var dengan type inference
var numbers = [1, 2, 3, 4, 5];

// Akses element
print(fruits[0]); // Apple
print(fruits[1]); // Banana

// Operasi list
fruits.add('Mango'); // Tambah di akhir
fruits.insert(0, 'Strawberry'); // Tambah di index 0
fruits.remove('Banana'); // Hapus by value
fruits.removeAt(2); // Hapus by index

// Properties
print(fruits.length); // Jumlah element
print(fruits.isEmpty); // Apakah kosong
print(fruits.first); // Element pertama
print(fruits.last); // Element terakhir

// Loop
for (var fruit in fruits) {
    print(fruit);
}
```

1.3 Maps (Dictionary/Object)

Map adalah kumpulan key-value pairs. Seperti dictionary atau object di bahasa lain.

```
// Create map
Map<String, dynamic> person = {
    'name': 'Budi',
    'age': 25,
    'email': 'budi@example.com',
    'isStudent': true,
};

// Akses value
print(person['name']); // Budi
print(person['age']); // 25

// Update value
person['age'] = 26;

// Add new key
person['phone'] = '08123456789';

// Remove key
person.remove('email');

// Check key exists
```

```

if (person.containsKey('name')) {
  print('Name exists');
}

// Loop
person.forEach((key, value) {
  print('$key: $value');
});

```

💡 Untuk Todo App:

```

Map<String, dynamic> task = {
  'id': '1',
  'title': 'Belajar Flutter',
  'description': 'Selesaikan tutorial',
  'isCompleted': false,
  'createdAt': DateTime.now(),
};

```

1.4 Functions

Function adalah blok kode yang bisa dipanggil berulang kali.

```

// Basic function
void sayHello() {
  print('Hello!');
}

// Function dengan parameter
void greet(String name) {
  print('Hello, $name!');
}

// Function dengan return value
int add(int a, int b) {
  return a + b;
}

// Arrow function (shorthand)
int multiply(int a, int b) => a * b;

// Optional parameters dengan []
void printInfo(String name, [int? age]) {
  print('Name: $name');
  if (age != null) {
    print('Age: $age');
  }
}

// Named parameters dengan {}
void createUser({
  required String name, // required = wajib diisi
  String? email,        // optional

```

```

    int age = 18,           // default value
  }) {
    print('Creating user: $name, $email, $age');
  }

// Cara panggil
createUser(name: 'Budi', email: 'budi@example.com');
createUser(name: 'Ani', age: 25);

```

1.5 Null Safety

Dart punya null safety - mencegah error karena null values.

```

// Non-nullable (tidak boleh null)
String name = 'Budi';
// name = null; // ERROR!

// Nullable (boleh null) - pakai tanda ?
String? email; // Default value = null
email = 'budi@example.com';
email = null;   // OK!

// Null check sebelum pakai
if (email != null) {
  print(email.length);
}

// Null-aware operators
String? username;
print(username ?? 'Guest'); // Jika null, pakai 'Guest'
username?.toUpperCase();    // Hanya panggil jika tidak null

// Null assertion (!) - paksa treat as non-null
String definitelyHasValue = username!; // Crash jika null!

```

💡 **Best practice:** Hindari ! kecuali yakin 100% tidak null.

1.6 Hands-on: Dart Playground

Mari coba Dart di DartPad online!

Langkah 1: Buka <https://dartpad.dev>

Langkah 2: Copy code ini dan run:

```

void main() {
  // Test variables
  var appName = 'Todo App';
  int totalTasks = 5;

  print('Welcome to $appName!');
  print('You have $totalTasks tasks');
}

```

```
// Test list
List<String> tasks = [
    'Belajar Dart',
    'Build UI',
    'Add Controller',
];

print('\nYour tasks:');
for (int i = 0; i < tasks.length; i++) {
    print('${i + 1}. ${tasks[i]}');
}

// Test map
Map<String, dynamic> task = {
    'title': 'Complete Workshop',
    'isCompleted': false,
    'priority': 'high',
};

print('\nCurrent task:');
print('Title: ${task['title']}');
print('Status: ${task['isCompleted']} ? 'Done' : 'Pending'}');

// Test function
int pendingCount = countPending(tasks);
print('\nPending tasks: $pendingCount');
}

int countPending(List<String> tasks) {
    return tasks.length;
}
```

Langkah 3: Coba modifikasi: - Tambah task baru ke list - Ubah isCompleted jadi true - Buat function baru untuk hitung completed tasks

 **Screenshot hasil run dan share ke grup!**

Bagian 2: Controllers & State Management (35 menit)

Di sesi sebelumnya, kita hardcode data di page. Sekarang kita pindahkan logic ke **Controller**.

2.1 What is Controller?

Controller adalah class yang: - Menyimpan **state** (data) aplikasi - Mengelola **business logic** (add, edit, delete) - **Memisahkan** UI dari logic (clean code!)

Analogi: - **Page** = Layar TV (hanya tampilkan) - **Controller** = Remote TV (kontrol apa yang ditampilkan)

2.2 Generate TodoHomeController

Controller sudah di-generate otomatis saat kita buat TodoHomePage di sesi 2. Tapi kita perlu modifikasi isinya.

Edit file: `lib/app/controllers/todo_home_controller.dart`

```
import 'package:flutter/material.dart';
import 'package:nylo_framework/nylo_framework.dart';

class TodoHomeController extends Controller {
  // State: List tasks (in-memory)
  List<Map<String, dynamic>> tasks = [];

  @override
  construct(BuildContext context) {
    super.construct(context);

    // Initialize with sample data
    tasks = [
      {
        'id': '1',
        'title': 'Belajar Flutter',
        'description': 'Selesaikan tutorial Flutter basics',
        'isCompleted': false,
        'createdAt': DateTime.now(),
      },
      {
        'id': '2',
        'title': 'Setup Project Nylo',
        'description': 'Install dan setup Nylo framework',
        'isCompleted': true,
        'createdAt': DateTime.now(),
      },
      {
        'id': '3',
        'title': 'Build ToDo App',
        'description': 'Buat aplikasi ToDo sederhana',
        'isCompleted': false,
        'createdAt': DateTime.now(),
      },
    ];

    // Helper getters
    int get totalTasks => tasks.length;

    int get completedTasks => tasks.where((t) => t['isCompleted'] ==
      true).length;

    int get pendingTasks => tasks.where((t) => t['isCompleted'] ==
      false).length;

    // Add more methods here (di bagian 3)
  }
}
```

💡 **Penjelasan:** - `construct()` dipanggil saat controller pertama kali dibuat - `tasks` adalah list yang menyimpan semua task - Getter methods untuk hitung stats (`total`, `completed`, `pending`)

2.3 Connect Controller ke TodoHomePage

Sekarang kita ubah `TodoHomePage` untuk pakai data dari controller.

Edit file: `lib/resources/pages/todo_home_page.dart`

Ubah bagian `_buildTaskList()`:

```
Widget _buildTaskList() {  
  // Ambil controller  
  final controller = widget.controller;  
  
  // Ambil tasks dari controller (bukan hardcode lagi!)  
  List<Map<String, dynamic>> tasks = controller.tasks;  
  
  return ListView.builder(  
    padding: EdgeInsets.all(16),  
    itemCount: tasks.length,  
    itemBuilder: (context, index) {  
      final task = tasks[index];  
      return _buildTaskCard(task);  
    },  
  );  
}
```

Ubah bagian `_buildStatsCard()`:

```
Widget _buildStatsCard() {  
  // Ambil controller  
  final controller = widget.controller;  
  
  // Ambil stats dari controller  
  int totalTasks = controller.totalTasks;  
  int completedTasks = controller.completedTasks;  
  int pendingTasks = controller.pendingTasks;  
  
  return Container(  
    margin: EdgeInsets.all(16),  
    padding: EdgeInsets.all(16),  
    decoration: BoxDecoration(  
      color: Colors.blue.shade50,  
      borderRadius: BorderRadius.circular(12),  
    ),  
    child: Row(  
      mainAxisAlignment: MainAxisAlignment.spaceAround,  
      children: [  
        _buildStatItem('Total', totalTasks, Colors.blue),  
        _buildStatItem('Done', completedTasks, Colors.green),  
        _buildStatItem('Pending', pendingTasks, Colors.orange),  
      ],  
    ),  
  );  
}
```

```

    ),
  );
}

Widget _buildStatItem(String label, int count, Color color) {
  return Column(
    children: [
      Text(
        count.toString(),
        style: TextStyle(
          fontSize: 28,
          fontWeight: FontWeight.bold,
          color: color,
        ),
      ),
      SizedBox(height: 4),
      Text(
        label,
        style: TextStyle(
          fontSize: 14,
          color: Colors.grey.shade700,
        ),
      ),
    ],
  );
}

```

Hot reload dan lihat hasilnya! Seharusnya sama seperti sebelumnya.

2.4 Understanding setState()

setState() adalah cara memberitahu Flutter untuk **rebuild UI** setelah data berubah.

Analogi: - Data berubah = Remote TV mengganti channel - setState() = Perintah layar TV untuk refresh tampilan

Contoh:

```

void toggleComplete(String taskId) {
  // Cari task by id
  final taskIndex = tasks.indexWhere((t) => t['id'] == taskId);

  if (taskIndex != -1) {
    // Update isCompleted
    tasks[taskIndex]['isCompleted'] = !tasks[taskIndex]
      ['isCompleted'];

    // WAJIB panggil setState agar UI update!
    setState(() {});
  }
}

```

✗ Tanpa setState(): - Data berubah di memory - UI tidak update (masih tampilan lama)

✨ Bagian 3: In-Memory CRUD Operations (30 menit)

Sekarang kita implementasi Create, Read, Update, Delete di controller!

3.1 Create Task

Tambahkan method di `todo_home_controller.dart`:

```
/// Add new task
void addTask({
  required String title,
  String description = '',
}) {
  // Generate simple ID (nanti akan pakai UUID atau dari database)
  String newId = DateTime.now().millisecondsSinceEpoch.toString();

  // Create task object
  Map<String, dynamic> newTask = {
    'id': newId,
    'title': title,
    'description': description,
    'isCompleted': false,
    'createdAt': DateTime.now(),
  };

  // Add to list
  tasks.insert(0, newTask); // Insert di awal list

  // Update UI
  setState(() {});

  print('Task added: $title');
}
```

Update `AddTaskPage` untuk panggil method ini:

Edit file: `lib/resources/pages/add_task_page.dart`

Ubah method `_handleSave()`:

```
void _handleSave() {
  String title = titleController.text.trim();
  String description = descriptionController.text.trim();

  // Validasi
  if (title.isEmpty) {
    showToastNotification(
      context,
      title: 'Error',
      description: 'Task title tidak boleh kosong!',
    );
  }
}
```

```

        style: ToastNotificationStyleType.DANGER,
    );
    return;
}

// Get TodoHomeController
final homeController =
    nyNavigator.navigator.getControllerOfType<TodoHomeController>
    ();

if (homeController != null) {
    // Call addTask method
    homeController.addTask(
        title: title,
        description: description,
    );

    // Show success message
    showToastNotification(
        context,
        title: 'Success',
        description: 'Task berhasil ditambahkan!',
        style: ToastNotificationStyleType.SUCCESS,
    );

    // Kembali ke home
    Future.delayed(Duration(seconds: 1), () {
        pop(context);
    });
}
}

```

🔥 **Test: Run app, tambah task baru, lihat muncul di list!**

3.2 Read Tasks

Read sudah kita implement dengan tasks list dan getter methods!

Method yang sudah ada:

```

int get totalTasks => tasks.length;
int get completedTasks => tasks.where((t) => t['isCompleted'] ==
    true).length;
int get pendingTasks => tasks.where((t) => t['isCompleted'] ==
    false).length;

```

Tambahan: Filter by status

```

/// Get tasks by status
List<Map<String, dynamic>> getTasksByStatus(bool isCompleted) {
    return tasks.where((t) => t['isCompleted'] == isCompleted).toList();
}

/// Get pending tasks only

```

```
List<Map<String, dynamic>> get pendingTasksList =>
    getTasksByStatus(false);

/// Get completed tasks only
List<Map<String, dynamic>> get completedTasksList =>
    getTasksByStatus(true);
```

3.3 Update Task

Tambahkan method di controller:

```
/// Update task
void updateTask({
    required String id,
    String? title,
    String? description,
    bool? isCompleted,
}) {
    // Cari task by id
    final taskIndex = tasks.indexWhere((t) => t['id'] == id);

    if (taskIndex == -1) {
        print('Task not found: $id');
        return;
    }

    // Update properties jika ada
    if (title != null) {
        tasks[taskIndex]['title'] = title;
    }

    if (description != null) {
        tasks[taskIndex]['description'] = description;
    }

    if (isCompleted != null) {
        tasks[taskIndex]['isCompleted'] = isCompleted;
    }

    // Update UI
    setState(() {});

    print('Task updated: $id');
}

/// Toggle complete status (shortcut)
void toggleComplete(String id) {
    final taskIndex = tasks.indexWhere((t) => t['id'] == id);

    if (taskIndex != -1) {
        bool currentStatus = tasks[taskIndex]['isCompleted'] ?? false;
        tasks[taskIndex]['isCompleted'] = !currentStatus;
        setState(() {});
    }
}
```

```
}  
}
```

Update DetailTaskPage untuk panggil toggleComplete:

Edit file: lib/resources/pages/detail_task_page.dart

Import controller di atas:

```
import '/app/controllers/todo_home_controller.dart';
```

Ubah button toggle complete:

```
// Toggle complete button  
SizedBox(  
  width: double.infinity,  
  height: 50,  
  child: ElevatedButton.icon(  
    onPressed: () {  
      // Get controller  
      final homeController =  
        nyNavigator.navigator.getControllerOfType<TodoHomeController>  
        ();  
  
      if (homeController != null && task != null) {  
        // Toggle status  
        homeController.toggleComplete(task!['id']);  
  
        // Update local task data untuk UI refresh  
        setState(() {  
          task!['isCompleted'] = !isCompleted;  
        });  
  
        // Show toast  
        showToastNotification(  
          context,  
          title: 'Success',  
          description: isCompleted  
            ? 'Task marked as pending'  
            : 'Task marked as completed',  
          style: ToastNotificationStyleType.SUCCESS,  
        );  
      }  
    },  
    icon: Icon(isCompleted ? Icons.undo : Icons.check),  
    label: Text(  
      isCompleted ? 'Mark as Pending' : 'Mark as Completed',  
      style: TextStyle(fontSize: 16),  
    ),  
    style: ElevatedButton.styleFrom(  
      backgroundColor: isCompleted ? Colors.orange : Colors.green,  
      foregroundColor: Colors.white,  
    ),  
  ),  
),  
,
```

🔥 **Test: Buka detail task, klik toggle button, kembali ke home, lihat status berubah!**

3.4 Delete Task

Tambahkan method di controller:

```
/// Delete task
void deleteTask(String id) {
  // Remove from list
  tasks.removeWhere((t) => t['id'] == id);

  // Update UI
  setState(() {});

  print('Task deleted: $id');
}
```

Update DetailTaskPage untuk implement delete dengan confirmation:

Ubah action button delete di AppBar:

```
IconButton(
  icon: Icon(Icons.delete),
  onPressed: () {
    _showDeleteConfirmation();
  },
),
```

Tambahkan method confirmation dialog:

```
void _showDeleteConfirmation() {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: Text('Delete Task?'),
      content: Text('Are you sure you want to delete this task? This action cannot be undone.'),
      actions: [
        TextButton(
          onPressed: () {
            pop(context); // Close dialog
          },
          child: Text('Cancel'),
        ),
        TextButton(
          onPressed: () {
            _handleDelete();
            pop(context); // Close dialog
          },
          style: TextButton.styleFrom(foregroundColor: Colors.red),
          child: Text('Delete'),
        ),
      ],
    ),
  ),
}
```

```

    );
}

void _handleDelete() {
    // Get controller
    final homeController =
        nyNavigator.navigator.getControllerOfType<TodoHomeController>
        ();

    if (homeController != null && task != null) {
        // Delete task
        homeController.deleteTask(task!['id']);

        // Show toast
        showToastNotification(
            context,
            title: 'Deleted',
            description: 'Task has been deleted',
            style: ToastNotificationStyleType.WARNING,
        );

        // Kembali ke home
        Future.delayed(Duration(milliseconds: 500), () {
            pop(context);
        });
    }
}

```

🔥 **Test: Delete task, confirm, lihat task hilang dari list!**

🎯 Testing Full CRUD

Mari test semua operasi CRUD yang sudah kita buat!

Test Checklist:

Create: - [] Klik FAB (+) - [] Isi form dengan title dan description - [] Klik Save - [] Task baru muncul di list (paling atas) - [] Stats card update (Total bertambah 1)

Read: - [] List tasks tampil semua - [] Stats card menampilkan count yang benar - [] Scroll list works

Update: - [] Klik task untuk buka detail - [] Klik toggle complete button - [] Status berubah (badge dan icon) - [] Kembali ke home, status updated - [] Stats card update (Done/Pending berubah)

Delete: - [] Buka detail task - [] Klik delete icon - [] Confirmation dialog muncul - [] Klik Delete - [] Task hilang dari list - [] Stats card update (Total berkurang 1)

Bonus: Improve UX

Empty State

Tambahkan empty state ketika tasks kosong.

Di `todo_home_page.dart`, `update_buildTaskList()`:

```
Widget _buildTaskList() {
  final controller = widget.controller;
  List<Map<String, dynamic>> tasks = controller.tasks;

  // Empty state
  if (tasks.isEmpty) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(Icons.inbox, size: 80, color: Colors.grey.shade300),
          SizedBox(height: 16),
          Text(
            'No tasks yet!',
            style: TextStyle(
              fontSize: 20,
              color: Colors.grey.shade600,
              fontWeight: FontWeight.bold,
            ),
          ),
          SizedBox(height: 8),
          Text(
            'Tap the + button to add your first task',
            style: TextStyle(
              fontSize: 14,
              color: Colors.grey.shade500,
            ),
          ),
        ],
      ),
    );
  }

  return ListView.builder(
    padding: EdgeInsets.all(16),
    itemCount: tasks.length,
    itemBuilder: (context, index) {
      final task = tasks[index];
      return _buildTaskCard(task);
    },
  );
}
```

Loading Indicator

Untuk simulasi loading (nanti akan real saat pakai database):

```
// Di controller
bool isLoading = false;

void addTask({required String title, String description = ''}) {
  setState(() {
    isLoading = true;
  });

  // Simulate async operation
  Future.delayed(Duration(milliseconds: 500), () {
    String newId = DateTime.now().millisecondsSinceEpoch.toString();

    Map<String, dynamic> newTask = {
      'id': newId,
      'title': title,
      'description': description,
      'isCompleted': false,
      'createdAt': DateTime.now(),
    };

    tasks.insert(0, newTask);

    setState(() {
      isLoading = false;
    });
  });
}
```



Common Issues & Solutions

Issue 1: “setState() called after dispose”

Symptom: Error saat navigate setelah operasi **Solution:** - Check if widget still mounted before setState

```
if (mounted) {
  setState(() {});
}
```

Issue 2: “Controller is null”

Symptom: getControllerOfType returns null **Solution:** - Pastikan TodoHomeController sudah di-attach ke TodoHomePage - Check navigation stack - controller harus ada di stack

Issue 3: “List not updating after add/delete”

Symptom: Data berubah tapi UI tidak update **Solution:** - Pastikan panggil `setState()` `{}` setelah modify data - Check if using correct controller instance

Issue 4: “Task ID duplicate”

Symptom: Multiple tasks dengan ID sama **Solution:** - Gunakan timestamp atau UUID untuk generate unique ID

```
import 'package:uuid/uuid.dart';  
String newId = Uuid().v4();
```



Key Takeaways

- ✅ **Dart Fundamentals:** - Variables: `var`, `final`, `const` - Collections: `List`, `Map` - Functions: regular dan arrow functions - Null safety: `?`, `??`, `?.`
 - ✅ **Controllers:** - Memisahkan UI logic dari business logic - Menyimpan state aplikasi - Methods untuk CRUD operations - `setState()` untuk trigger UI rebuild
 - ✅ **State Management:** - Data di controller, UI di page - `setState()` mandatory untuk update UI - Controller accessible via `widget.controller` atau `getControllerOfType`
 - ✅ **CRUD Operations:** - **Create:** `add()`, `insert()` - **Read:** Direct access list, filter dengan `where()` - **Update:** Find by index/id, modify properties - **Delete:** `removeWhere()`, `removeAt()`
-



Resources

- **Dart Language Tour:** <https://dart.dev/language>
 - **Nylo Controllers:** <https://nylo.dev/docs/6.x/controllers>
 - **Flutter State Management:** <https://docs.flutter.dev/data-and-backend/state-mgmt/intro>
 - **Dart Collections:** <https://dart.dev/guides/libraries/library-tour#collections>
-



Navigation

← [Sesi 2 - UI & Navigation](#) | [Sesi 4 - Model & Data Persistence](#) →

Workshop Material - Simple ToDo List with Flutter Nylo | Dokumentasi Sesi 3 - State Management & Local Data | Terakhir diperbarui: November 21, 2025